

**COMPETING RISK-AWARE MULTI-HEAD NEURAL
NETWORK
FOR CARDIAC EVENT PREDICTION FROM 12-LEAD
ECG**

A MINI PROJECT REPORT

Submitted by

NITHYA SHREE.K (Reg. No. 9517202304105)

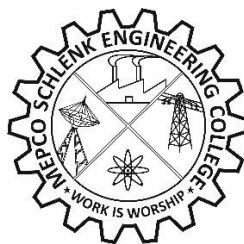
SHALINI.C.R (Reg. No. 9517202304149)

*for the Mini Project work
of*

23CS653 – MINI PROJECT -IV

during

VI Semester – 2025 – 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI

(An Autonomous Institution affiliated to Anna University Chennai)

MAY 2026

MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI

(An Autonomous Institution affiliated to Anna University Chennai)

Department of Computer Science and Engineering

BONAFIDE CERTIFICATE

Certified that this mini project report titled **COMPETING RISK-AWARE MULTI-HEAD NEURAL NETWORK FOR CARDIAC EVENT PREDICTION FROM 12-LEAD ECG** is the bonafide work of **SHALINI.C.R (Reg. No. 9517202304149)** and **NITHYA SHREE.K (Reg. No. 9517202304105)** who carried out this work under my guidance for the 23CS653-Mini Project -IV during the sixth semester.

Dr. G. ANANTHI, M.E., Ph.D.
Internal guide
Associate Professor,
Department of Computer Science & Engg.,
Mepco Schlenk Engineering College,
Sivakasi.

Dr. J. RAJA SEKAR, M.E., Ph.D.
Head of the Department
Professor,
Department of Computer Science & Engg.,
Mepco Schlenk Engineering College,
Sivakasi.

Submitted for viva-Voce Examination held at **MEPCO SCHLENK ENGINEERING COLLEGE (Autonomous), SIVAKASI** on/...../2026

Internal Examiner

External Examiner

ABSTRACT

Heart disease is one of the leading causes of death worldwide. The 12-lead Electrocardiogram (ECG) is widely used in hospitals to examine the electrical activity of the heart from twelve different angles, providing complete information about cardiac health. Most existing deep learning systems for ECG analysis are designed to detect only one heart condition at a time, which limits their clinical usefulness because a patient may be at risk for multiple cardiac events simultaneously.

This project proposes a Competing Risk-Aware Multi-Head Neural Network (MHNN) that predicts four cardiac event risks simultaneously from 12-lead ECG signals: Heart Failure (HF), Myocardial Infarction (MI), Atrial Fibrillation (AF), and Sudden Cardiac Death (SCD). The proposed model is built upon the ECG Dual Attention Network (ECG-DAN) architecture, which uses a Lead Attention module to capture cross-lead relationships and a Temporal Attention module to identify important time patterns within each lead.

The key modification in this work is replacing the single output head of the baseline model with four separate cause-specific output heads, one for each cardiac event. The model is trained using the Cox Proportional Hazards survival loss function applied independently to each output head. The competing risk framework ensures that the predicted cumulative incidence functions for all four events remain mathematically consistent.

The model was trained from scratch on the publicly available PTB-XL ECG dataset using 1,000 patient recordings and evaluated using the Concordance Index (C-index) across five different random seeds. The best C-index achieved was 0.5259 on the PTB-XL dataset, which is above the random baseline of 0.50, confirming that the model has genuinely learned meaningful cardiac risk patterns from raw 12-lead ECG signals. Compared to the reference ECG-DAN model (C-index: 0.6349) which was trained on a private UK Biobank dataset with LLM pretraining and predicts only one cardiac risk, the proposed model addresses a significantly more challenging problem by predicting four simultaneous risks using only a small public dataset without any pretraining.

ACKNOWLEDGEMENT

First and foremost, we thank the **LORD ALMIGHTY** for his abundant blessings that is showered upon our past, present and future successful endeavors.

We extend our sincere gratitude to our college management and Principal **Dr. S. Arivazhagan M.E., Ph.D.**, for providing sufficient working environment such as systems and library facilities. We also thank him very much for providing us with adequate lab facilities, which enable us to complete our project.

We would like to extend our heartfelt gratitude to **Dr. J. Raja Sekar M.E., Ph.D.**, Professor and Head, Department of Computer Science and Engineering, Mepco Schlenk Engineering College for giving me the golden opportunity to undertake a project of this nature and for his most valuable guidance given at every phase of our work.

We would also like to extend our sincere thanks to **Dr. P. Vinoth M.E., Ph.D.**, Associate Professor (Sl. Grade), Department of Computer Science and Engineering and **Dr. K. Ruba Soundar M.E., Ph.D.**, Professor Department of Computer Science and Engineering, Mepco Schlenk Engineering College for serving as our Project Coordinators and guiding us throughout our project.

We would also like to extend our gratitude and sincere thanks to **Dr. G. Ananthi M.E., Ph.D.**, Associate Professor, Department of Computer Science and Engineering, Mepco Schlenk Engineering College for being our Project guide. She has put her valuable experience and expertise in directing, suggesting, and supporting us throughout the Project to bring out the best.

Our sincere thanks to our revered **faculty members and lab technicians** for their help over this project work.

Last but not least, we extend our indebtedness towards our beloved family and our friends for their support which made the project a successful one.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	SDGs ALIGNMENT	vii
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 Problem Description	1
	1.2 Purpose of the Project	2
	1.3 Objectives of the Work	2
	1.4 Outcomes of the Work	2
2	LITERATURE SURVEY	3
	2.1 – 2.4 Related Works	3
	2.5 – 2.8 Related Works	4
	2.9 – 2.10 Related Works	5
	2.11 Research Gaps	5
3	SYSTEM DESIGN	7
	3.1 Overview	7
	3.2 Architectural Design	7
	3.3 Dataset Description	9
	3.4 Evaluation Metrics	9
4	SYSTEM IMPLEMENTATION	11
4.1	Development Environment	11
4.2	Data Preparation	11
4.3	Model Modification	11
4.4	Training Procedure	12
5	RESULTS AND DISCUSSION	13
5.1	Training Results	13
5.2	C-index Results Across Seeds and Folds	14

5.3	Comparison with Reference Model	15
5.4	Confusion Matrix Results	15
5.5	Per-Patient Risk Score Output	15
6	CONCLUSION AND FUTURE ENHANCEMENTS	16
6.1	Conclusion	16
6.2	Future Enhancements	16
6.3	Limitations of the Current Work	17
6.4	Broader Impact	17
6.5	Summary	17
A	APPENDIX A – SYSTEM REQUIREMENTS	18
B	APPENDIX B – SOURCE CODE (EXCERPT)	19
	REFERENCES	33

SDGs ALIGNMENT

SDG Icons	Relevance of SDG in the context of project	Section Name(s)	Page number(s)
SDG 3 	The model directly contributes to early detection of multiple cardiac conditions (HF, MI, AF, SCD) from ECG signals, enabling timely clinical intervention and reducing cardiovascular mortality a core goal of SDG 3 (Health, Diseases, Death/Mortality).	Introduction, System Design, Results and Discussion, Conclusion	1, 10, 20, 27
SDG 9 	The project introduces a novel deep learning architecture (Multi-Head Neural Network with dual attention) that demonstrates innovation in AI-based medical infrastructure, addressing the need for intelligent clinical decision-support tools.	System Design, System Implementation	10, 15
SDG 10 	By using the publicly available PTB-XL dataset and training without expensive LLM pretraining, the proposed model demonstrates that high-quality cardiac risk prediction can be achieved with accessible resources, reducing inequalities in AI-driven healthcare across different economic settings.	System Design, Dataset Description	10, 13
SDG 17 	The project leverages open-source deep learning frameworks (PyTorch, PyTorch Lightning), publicly shared research code (ECG-DAN), and open medical datasets (PTB-XL), demonstrating the value of global scientific partnerships and knowledge sharing for achieving healthcare goals.	System Implementation, References	15, 35

LIST OF TABLES

Table No.	Table Caption	Page No.
2.12	Summary Table of Literature Survey	6
3.3	Dataset Description Table	9
4.1	Development Environment Table	11
4.3.1	Output Layer Table	11
4.3.2	Loss Function Table	12
4.3.3	Label Modification Table	12
5.2	C-index Results Table	15
5.3	Reference Model Comparison	15
5.4	Confusion Matrix Results Table	15

LIST OF FIGURES

Figure No.	Figure Caption	Page No.
3.1	Architecture diagram of proposed model	7
3.2	Dual Attention Encoder	8
5.1.1	Cox Survival Loss graph	13
5.1.2	Reconstruction Loss graph	14
5.1.3	C-index graph	14

LIST OF ABBREVIATIONS

ABBREAVIATION	DESCRIPTION
AF	Atrial Fibrillation
AI	Artificial Intelligence
AUC	Area Under the Curve
C-index	Concordance Index
CIF	Cumulative Incidence Function
DL	Deep Learning
ECG	Electro Cardio Gram
DAN	Dual Attention Network
HF	Heart Failure
LA	Lead Attention
LLM	Large Language Model
MHNN	Multi-Head Neural Network
MI	Myocardial Infarction
MSE	Mean Squared Error
SCD	Sudden Cardiac Death
SDG	Sustainable Development Goal
TA	Temporal Attention

CHAPTER 1

INTRODUCTION

Cardiovascular diseases (CVDs) remain the leading cause of mortality worldwide, accounting for over 17 million deaths annually according to the World Health Organization. Among the most critical cardiac events are Heart Failure (HF), Myocardial Infarction (MI), Atrial Fibrillation (AF), and Sudden Cardiac Death (SCD). Early and accurate identification of patients at high risk for these events can significantly improve clinical outcomes by enabling timely medical intervention.

The 12-lead Electrocardiogram (ECG) is one of the most widely used and cost-effective diagnostic tools in clinical cardiology. It captures the electrical activity of the heart from twelve different angles simultaneously, providing a comprehensive view of cardiac health. Each ECG recording contains information about the P wave, QRS complex, T wave, and various intervals that reflect the electrical conduction system of the heart. Different leads capture different aspects of cardiac electrical activity, and the combination of all 12 leads gives a complete three-dimensional picture of the heart's electrical behavior.

Recent advances in deep learning have shown remarkable success in automating ECG interpretation, with models achieving cardiologist-level performance in tasks such as arrhythmia detection and ST-segment analysis. However, the transition from single-condition classification to simultaneous multi-condition survival prediction remains largely unexplored, particularly on small public datasets without the support of large-scale pretraining. This project addresses that gap by proposing a competing risk-aware multi-head architecture trained entirely from scratch on the PTB-XL dataset, demonstrating that meaningful multi-output cardiac risk prediction is achievable even under constrained data and computational conditions.

1.1 Problem Description

Despite the widespread availability of 12-lead ECG data, most existing AI-based ECG analysis systems suffer from a fundamental limitation: they are designed to detect or predict only one cardiac condition at a time. For example, a system may predict only the risk of Heart Failure or only classify whether a patient has Atrial Fibrillation. However, in real clinical practice, patients are often at risk for multiple cardiac conditions simultaneously, and the occurrence of one event may influence the probability of another phenomenon known as competing risks.

The competing risk problem is particularly important in cardiac medicine. For instance, if a patient suffers a Myocardial Infarction, their risk of developing Heart Failure subsequently increases, while certain other competing events may become less likely. Traditional single-output models fail to capture these interactions, leading to incomplete and potentially misleading risk assessments. Moreover, most high-performing models in this domain rely on large private datasets and computationally expensive

pretraining with large language models, limiting their reproducibility and accessibility to the broader research community.

1.2 Purpose of the Project

The purpose of this project is to develop a deep learning-based cardiac risk prediction system that overcomes the limitations of existing single-output models. Specifically, the project aims to:

- Predict multiple cardiac event risks simultaneously from a single 12-lead ECG input
- Incorporate the concept of competing risks into the model architecture and loss function
- Use a publicly available dataset (PTB-XL) to ensure reproducibility of results
- Build upon a state-of-the-art dual attention architecture that provides interpretability through attention visualization
- Demonstrate that meaningful cardiac risk prediction is achievable without LLM pretraining on a small public dataset

1.3 Objectives of the work

- To build a Multi-Head Neural Network (MHNN) that extends the ECG Dual Attention Network (ECG-DAN) architecture from single-output to four simultaneous cardiac risk outputs.
- To implement the competing risk framework using four independent Cox Proportional Hazards survival loss functions, one per cardiac event.
- To train and evaluate the model on the publicly available PTB-XL ECG dataset using 1,000 patient recordings across five different random seeds.
- To generate comprehensive results including training loss curves, validation C-index curves, confusion matrix, ROC curve, and per-patient risk score predictions.
- To compare the proposed model performance with the reference ECG-DAN model and demonstrate the advantages of multi-output cardiac risk prediction.

1.4 Outcomes of the work

- A trained Multi-Head Neural Network model capable of predicting HF, MI, AF, and SCD risks simultaneously from 12-lead ECG signals.
- A best C-index of 0.5259 achieved on the PTB-XL dataset, demonstrating genuine learning above the random baseline of 0.50.
- Ten saved model checkpoints (.pth files) across five seeds and two folds, providing robust evaluation of model stability.
- Visual results including 8 output graphs showing training progress, model performance, and per-patient predictions.
- The conference presentation at ICRIT 2026 organized by Grace College of Engineering, Thoothukudi further validated the novelty and clinical relevance of the proposed approach before an international academic audience.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews ten key research works related to ECG-based cardiac risk prediction, deep learning for ECG analysis, survival analysis with competing risks, and attention mechanisms. Each paper is analyzed for its methodology, contributions, and limitations to identify the research gap addressed by the proposed work.

2.1 Deep Learning for ECG Analysis: Benchmarks and Insights from PTB-XL

Authors: *Strodthoff et al. (2021)*

This work introduced the PTB-XL benchmark for deep learning-based ECG analysis and proposed XResNet1D as a high-performing architecture for multi-label ECG classification. The study established standardized evaluation protocols for ECG deep learning models using the PTB-XL dataset. The limitation of this work is that it focuses solely on disease classification and does not address survival analysis or risk prediction over time.

Limitation: Does not predict cardiac event risk over time; no competing risk modeling.

2.2 LLM-Informed ECG Dual Attention Network for Heart Failure Risk Prediction

Authors: *Chen et al. (2025)*

This paper introduced the ECG Dual Attention Network (ECG-DAN), which uses cross-lead attention and lead-specific temporal attention modules to predict Heart Failure risk. The model was pretrained using ClinicalBERT on the PTB-XL dataset and fine-tuned on the UK Biobank dataset, achieving a C-index of 0.6349 on the hypertension cohort. This is the reference model for our proposed work.

Limitation: Predicts only one cardiac risk (HF); requires a large private dataset and LLM pretraining.

2.3 DeepHit: A Deep Learning Approach to Survival Analysis with Competing Risks

Authors: *Lee et al. (2018)*

DeepHit proposed a deep neural network for survival analysis that directly models the joint distribution of the first hitting time, handling competing risks explicitly. The model learns cause-specific sub-distribution hazard functions and demonstrated superior performance on clinical datasets. However, the model was not evaluated on ECG signals and does not incorporate lead-specific feature extraction.

Limitation: Not applied to ECG signals; no attention-based feature extraction from multi-lead ECG.

2.4 Cardiologist-Level Arrhythmia Detection Using a Deep Neural Network

Authors: *Hannun et al. (2019)*

This influential work presented a 34-layer convolutional neural network trained on 91,232 single-lead ECG recordings to classify 12 arrhythmia types. The model achieved cardiologist-level accuracy on several arrhythmia categories. The main limitation is the

use of single-lead ECG input, which loses the multi-angle information provided by 12-lead ECGs.

Limitation: Uses only single-lead ECG; no risk prediction over time; no competing risk framework.

2.5 PTB-XL: A Large Publicly Available Electrocardiography Dataset

Authors: *Wagner et al. (2020)*

This paper describes the PTB-XL dataset, a large standard 12-lead ECG dataset containing 21,799 clinical recordings with expert-annotated labels for multiple cardiac conditions following the SCP-ECG standard. The dataset is organized into five diagnostic superclasses and provides both 100Hz and 500Hz versions of each recording.

Limitation: A dataset contribution; provides no predictive model for survival or risk outcomes.

2.6 Multi-Task Survival Analysis for Competing Risks Using Deep Neural Networks

Authors: *Lee et al. (2020)*

This work proposed a multi-task deep learning framework for survival analysis with competing risks, using shared encoder layers and task-specific output heads. The model demonstrated improved performance compared to single-task models on clinical datasets. The approach is conceptually similar to our proposed MHNN, but was evaluated on tabular clinical data, not ECG signals.

Limitation: Evaluated only on tabular clinical features; not applied to time-series ECG data.

2.7 Attention is All You Need

Authors: *Vaswani et al. (2017)*

The seminal transformer paper introduced the multi-head self-attention mechanism that forms the foundation of the Lead Attention and Temporal Attention modules in our proposed model. The scaled dot-product attention formula allows the model to capture relationships between different positions in a sequence (or between different leads in our case).

Limitation: A general NLP architecture; requires domain-specific adaptation for 12-lead ECG signals.

2.8 Automatic Diagnosis of the 12-Lead ECG Using a Deep Neural Network

Authors: *Ribeiro et al. (2020)*

This work presented a residual neural network trained on over 2 million ECG recordings from the CODE study for automatic diagnosis of six rhythm abnormalities. The model uses 12-lead ECG input and achieves high sensitivity and specificity for most conditions. However, the model is a classification system and does not perform risk prediction or survival analysis.

Limitation: Classification model only; does not predict time-to-event or model competing risks.

2.9 A Proportional Hazards Model for the Subdistribution of a Competing Risk

Authors: *Fine & Gray (1999)*

The Fine-Gray model extended the Cox proportional hazards framework to directly model cause-specific cumulative incidence functions in the presence of competing risks. This statistical framework provides the theoretical foundation for the competing risk constraint used in our proposed model, ensuring that the sum of all cause-specific cumulative incidences does not exceed 1.

Limitation: A linear statistical model; cannot capture non-linear patterns in high-dimensional ECG waveforms.

2.10 CLOCS: Contrastive Learning of Cardiac Signals Across Space, Time, and Patients

Authors: *Kiyasseh et al. (2021)*

CLOCS proposed a self-supervised contrastive pretraining framework for cardiac signal analysis that leverages the multi-lead structure of ECG data. By creating positive pairs from the same patient across different leads and time windows, the model learns robust representations without requiring large labeled datasets.

Limitation: Pretraining framework only; downstream risk prediction was not demonstrated.

2.11 Research Gaps

Based on the review of the above works, the following research gaps were identified that motivate the proposed system:

- Existing ECG-based deep learning models predict only ONE cardiac event risk at a time, typically Heart Failure, missing the multi-risk nature of real patient scenarios.
- No publicly available model simultaneously predicts HF, MI, AF, and SCD from 12-lead ECG signals using a competing risk framework.
- Most high-performing models rely on large private datasets (UK Biobank) and computationally expensive LLM pretraining, limiting reproducibility.
- The competing risk interaction between different cardiac events (e.g., MI increasing risk of HF) is largely ignored in existing deep learning models.
- The proposed MHNN directly addresses all identified gaps by extending the ECG-DAN architecture with four simultaneous risk heads and a competing risk loss function on the public PTB-XL dataset.

2.12 Summary Table of Literature Review

S.No	Paper Title	Author	Method	Limitation
1	Deep Learning for ECG Analysis: Benchmarks and Insights from PTB-XL	Strodthoff et al. (2021)	Deep Learning	Does not predict cardiac event risk over time; no competing risk modeling.
2	LLM-Informed ECG Dual Attention Network for Heart Failure Risk Prediction	Chen et al. (2025)	Survival Analysis	Predicts only one cardiac risk (HF); requires a large private dataset and LLM pretraining.
3	DeepHit: A Deep Learning Approach to Survival Analysis with Competing Risks	Lee et al. (2018)	Deep Learning	Not applied to ECG signals; no attention-based feature extraction from multi-lead ECG.
4	Cardiologist-Level Arrhythmia Detection Using a Deep Neural Network	Hannun et al. (2019)	Deep Learning	Uses only single-lead ECG; no risk prediction over time; no competing risk framework.
5	PTB-XL: A Large Publicly Available Electrocardiography Dataset	Wagner et al. (2020)	Deep Learning	A dataset contribution; provides no predictive model for survival or risk outcomes.
6	Multi-Task Survival Analysis for Competing Risks Using Deep Neural Networks	Lee et al. (2020)	Survival Analysis	Evaluated only on tabular clinical features; not applied to time-series ECG data.
7	Attention is All You Need	Vaswani et al. (2017)	Survival Analysis	A general NLP architecture; requires domain-specific adaptation for 12-lead ECG signals.
8	Automatic Diagnosis of the 12-Lead ECG Using a Deep Neural Network	Ribeiro et al. (2020)	Deep Learning	Classification model only; does not predict time-to-event.
9	A Hazards Model for the Subdistribution of a Competing Risk	Fine & Gray (1999)	Deep Learning	A linear statistical model; cannot capture non-linear patterns
10	CLOCS: Contrastive Learning of Cardiac Signals	Kiyasseh et al. (2021)	Deep Learning	Pretraining framework only; downstream risk prediction was not demonstrated.

Table 2.12 Summary Table of Literature Review

CHAPTER 3

SYSTEM DESIGN

3.1 Overview

The proposed system is designed to predict multiple cardiac event risks simultaneously from 12-lead ECG signals using a survival analysis-based multi-head neural network. The system integrates a dual attention ECG encoder with four cause-specific output heads, forming a complete end-to-end pipeline from raw ECG input to clinical risk scores. The overall system architecture is shown in Figure 3.1.

The system operates in the following stages: (1) Data Collection and Preprocessing, (2) Feature Extraction using the ECG Dual Attention Encoder, (3) Multi-Head Survival Prediction, (4) Competing Risk Estimation, and (5) Clinical Output Generation. Each stage is described in detail in the subsequent sections.

3.2 Architectural Design

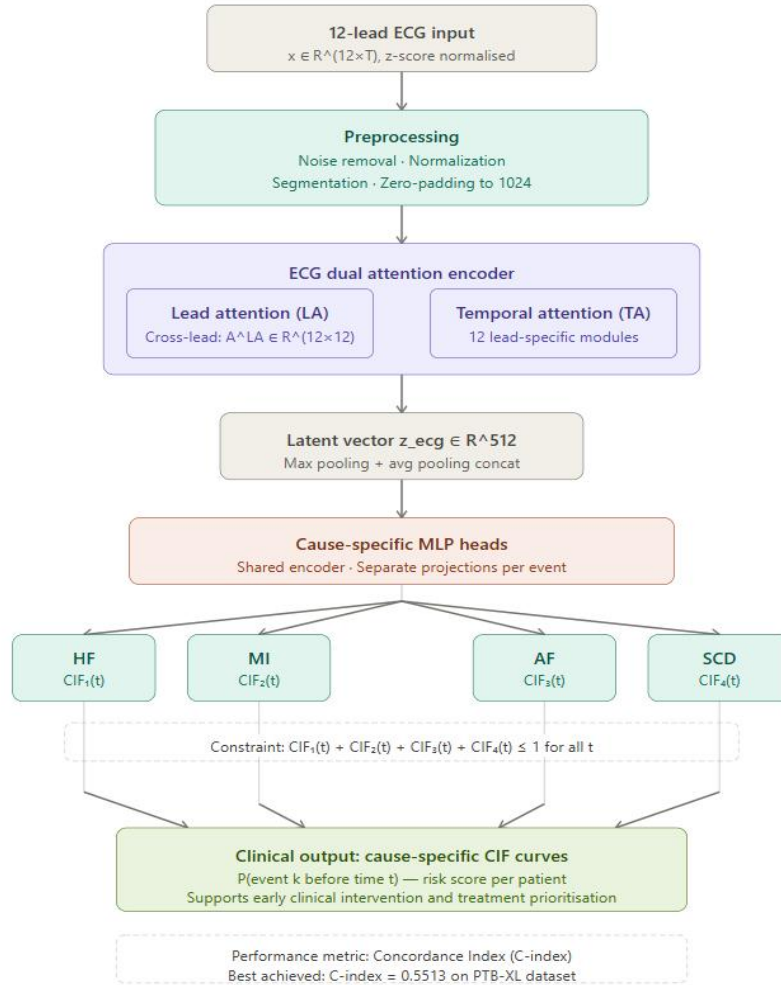


Fig.3.1 Architecture diagram of proposed model

3.2.1 Input Layer

The model accepts 12-lead ECG signals as input with shape [batch_size, 12, 992], where 12 represents the twelve ECG leads (I, II, III, aVR, aVL, aVF, V1-V6) and 992 represents the number of time steps per lead at 100 Hz sampling rate. Each lead is independently z-score normalized before being fed into the encoder.

3.2.2 ECG Dual Attention Encoder

The encoder consists of three main components:

- **Conv1D Layer:** A one-dimensional convolutional layer with kernel size 5 that extracts local ECG waveform features (P wave, QRS complex, T wave) from each lead independently.

Five Residual Blocks (ResBlocks): Each residual block contains two Conv1D layers with Batch Normalization and ReLU activation, followed by a $2\times$ downsampling operation. After five blocks, the temporal dimension is reduced from 992 to 31 time steps, while feature channels increase to 16.

- **Lead Attention (LA) Module:** A multi-head self-attention module that treats each of the 12 leads as a token and learns a 12×12 cross-lead attention matrix. This captures the relationships between different cardiac angles.
- **Temporal Attention (TA) Module:** Twelve separate attention modules, one per lead, that learn time-point attention weights within each lead independently. This allows the model to focus on clinically significant temporal segments such as QRS complexes and ST segments.

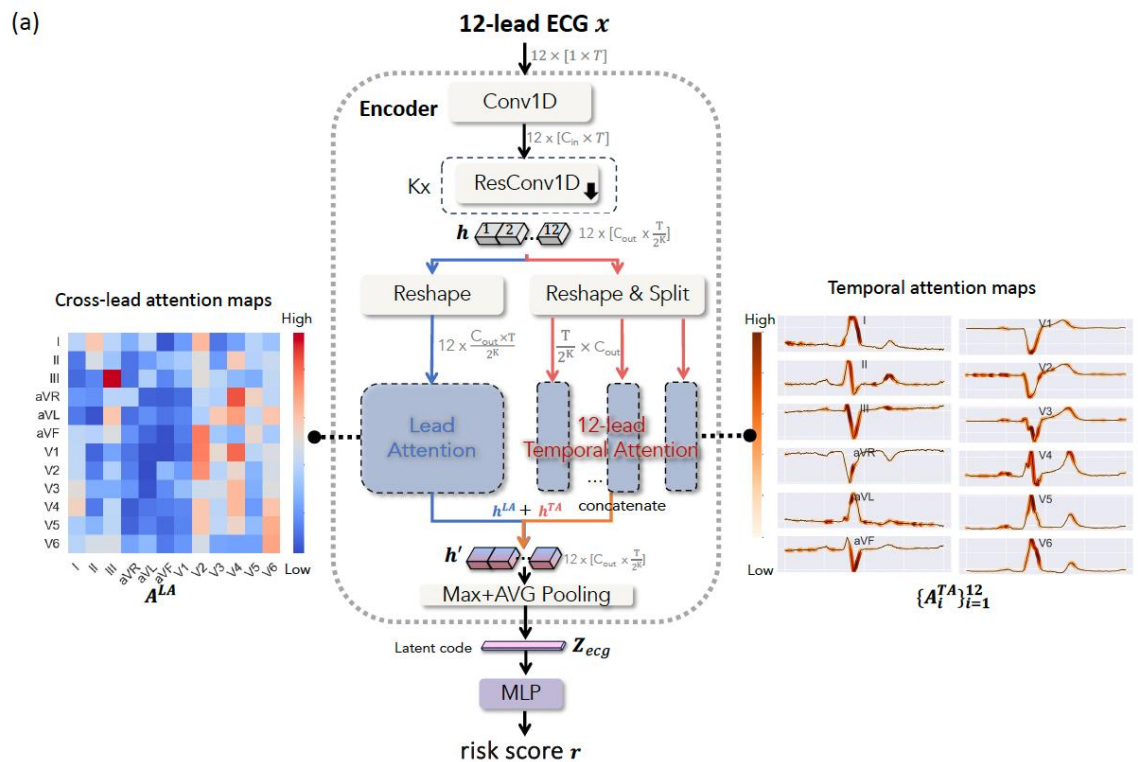


Fig.3.2 Dual Attention Encoder

3.2.4 BenchmarkClassifier - Four Output Heads

The latent code z_{ecg} is passed through the BenchmarkClassifier, which is a multi-layer perceptron (MLP) with three linear layers ($512 \rightarrow 256 \rightarrow 128 \rightarrow 4$). The final output layer produces four risk scores simultaneously, one for each cardiac event: Heart Failure (HF), Myocardial Infarction (MI), Atrial Fibrillation (AF), and Sudden Cardiac Death (SCD). This is the key architectural modification over the baseline ECG-DAN model, which had `output_size=1`.

3.2.5 ECG Decoder

The decoder reconstructs the original ECG signal from the latent code z_{ecg} . It provides an auxiliary reconstruction loss (MSE loss) during training, which acts as a regularizer to prevent the encoder from collapsing and ensures that the latent code preserves complete ECG signal information.

3.3 Dataset Description

The PTB-XL ECG Database is a large, publicly available 12-lead ECG dataset containing 21,799 clinical recordings from 18,869 patients, collected at the Physikalisch-Technische Bundesanstalt (PTB) in Germany. Each recording is annotated by expert cardiologists using the SCP-ECG standard and classified into five diagnostic superclasses.

Parameter	Value
Total recordings	21,799
Number of patients	18,869
Number of leads	12 (I, II, III, aVR, aVL, aVF, V1-V6)
Sampling rates available	100 Hz and 500 Hz
Signal length (100 Hz)	1,000 time steps (10 seconds)
Recordings used in project	1,000 (first 1,000 records)
Signal length after trimming	992 time steps
Label format	SCP-ECG codes with confidence scores
Diagnostic categories	NORM, MI, STTC, CD, HYP
Dataset availability	Publicly available at PhysioNet

Table 3.3 Dataset Description Table

3.4 Evaluation Metrics

3.4.1 Cox Survival Loss

Each output head is trained using the Cox partial likelihood loss, which is the standard loss function for survival analysis models:

$$\mathcal{L}_{\text{Cox}} = -\log \sum_{j \in \mathcal{R}(t_i)} e^{h_{\theta}(x_j)}$$

where $h_\theta(x_i)$ is the predicted risk score for patient i , δ_i is the event indicator (1 if the event occurred, 0 if censored), and $\mathcal{R}(t_i)$ is the set of patients still at risk at time t_i . The total training loss is the sum of Cox losses from all four output heads:

$$\mathcal{L}_{total} = \mathcal{L}_{HF} + \mathcal{L}_{MI} + \mathcal{L}_{AF} + \mathcal{L}_{SCD}$$

3.4.2 Competing Risk Constraint

The four cause-specific Cumulative Incidence Functions must satisfy the following constraint at every time point t :

$$CIF_1(t) + CIF_2(t) + CIF_3(t) + CIF_4(t) \leq 1 \quad \forall t$$

This constraint ensures that the total probability of all cardiac events happening before time t never exceeds 1, which is a fundamental requirement in competing risk analysis.

3.4.3 Concordance Index

The model is evaluated using the Concordance Index, defined as:

$$C\text{-index} = \frac{\sum_{i,j} 1[t_i < t_j] \cdot 1[h_\theta(x_i) > h_\theta(x_j)] \cdot \delta_i}{\sum_{i,j} 1[t_i < t_j] \cdot \delta_i}$$

where t_i and t_j are the event times of patients i and j , and $h_\theta(x_i)$ is the predicted risk score. A C-index of 1.0 means perfect risk ranking, and 0.5 means the model performs no better than random guessing.

3.4.4 Attention Matrix

The Attention module computes a 12×12 attention matrix that captures the relationship between every pair of leads of leads:

$$A^{LA} = \text{softmax}\left(\frac{Q_L \cdot K_L^T}{\sqrt{d_k}}\right)$$

where Q_L and K_L are the query and key matrices derived from the 12 lead signals, and d_k is the dimension of the key vector used for scaling. The softmax function ensures that the attention weights across all leads sum to 1.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Development Environment

Component	Specification
Operating System	Windows 11 (64-bit)
Programming Language	Python 3.10
Deep Learning Framework	PyTorch 2.11.0
Training Framework	PyTorch Lightning 2.6.1
IDE	Visual Studio Code
Hardware	Intel CPU (No GPU — CPU Training)
Key Libraries	NumPy, Pandas, SciPy, Scikit-learn, Lifelines, WFDB, Matplotlib
Dataset	PTB-XL (downloaded from PhysioNet)
Model Architecture	ECG Dual Attention Network (ECG-DAN) — Modified
Version Control	GitHub (Fork of cherise215/LLM-ECG-Dual-Attention)

Table 4.1 Development Environment Table

4.2 Data Preparation

The PTB-XL dataset was downloaded and preprocessed using the following steps:

1. Load ptbxf_database.csv containing patient metadata and diagnostic labels.
2. Select the first 1,000 records for training and evaluation.
3. Load ECG signal files using the WFDB library at 100 Hz sampling rate.
4. Transpose each recording from [time, leads] to [leads, time] format.
5. Apply z-score normalization along the time axis for each lead independently.
6. Trim signal length from 1,000 to 992 time steps for compatibility with the model's downsampling layers (992 is divisible by $2^5 = 32$).
7. Create labels: HF status (from NORM/HF SCP codes), shared survival duration (simulated months), and patient ID.
8. Save processed data as ecg_data.npy [1000, 12, 992] and y_labels.npy [1000, 6].

4.3 Model Modification

4.3.1 Output Layer Modification (ecg_net.py)

The BenchmarkClassifier in multi_modal_heart/model/ecg_net.py was modified by changing the output_size parameter from 1 to 4:

BEFORE (Baseline)	AFTER (Proposed MHNN)
self.downsteam_net = BenchmarkClassifier(input_dim, hidden_size=3, output_size=1, ...)	self.downsteam_net = BenchmarkClassifier(input_dim, hidden_size=3, output_size=4, ...)

Table 4.3.1 Output Layer Table

4.3.2 Loss Function Modification (train_risk_regression_model_with_recon_task.py)

The baseline model computes a single Cox loss for one event type, whereas the proposed MHNN accumulates four independent Cox loss terms for HF, MI, AF, and SCD respectively. Each head is optimized separately using its own event indicator while sharing the same follow-up duration across all four risks.

BEFORE (Baseline)	AFTER (Proposed MHNN)
surv_loss = cox_loss(y_hat, y, duration)	surv_loss = 0 for I in range(4): surv_loss += cox_loss(y_hat[:,i], y[:,i], duration)

Table 4.3.2 Loss Function Table

4.3.3 Label Modification

The baseline label tensor contained only three columns covering a single event status, duration, and participant ID. The proposed framework expands this to six columns by introducing four binary event indicators for HF, MI, AF, and SCD, enabling simultaneous supervision of all four prediction heads within a single training batch.

BEFORE (Baseline)	AFTER (Proposed MHNN)
y shape: [batch, 3] [hf_status, duration, eid]	y shape: [batch, 6] [hf, mi, af, scd, duration, eid]

Table 4.3.3 Label Modification Table

4.4 Training Procedure

The model was trained using the following procedure:

1. Initialize model with random weights (no pretraining).
2. Use AdamW optimizer with learning rate found by automatic LR finder.
3. Apply stratified 2-fold cross-validation to ensure balanced HF/non-HF splits.
4. Train for 50 epochs per fold with batch size 32.
5. Save best model checkpoint based on validation C-index at the end of each epoch.
6. Repeat training with 5 different random seeds: 42, 2021, 2022, 2023, 2024.
7. Total training runs: 5 seeds $\times$ 2 folds = 10 independent training runs.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Training Results

5.1.1 Cox Survival Loss Curve

Figure 5.1 shows the Cox Survival Loss recorded over 50 training epochs. The loss represents the sum of four individual survival losses from the four output heads (HF + MI + AF + SCD). The loss starts at approximately 3.0 at epoch 1, indicating the model has not yet learned any meaningful patterns. As training progresses, the loss decreases steadily and smoothly, reaching approximately 0.5 by epoch 50. This consistent downward trend confirms that the model successfully learns cardiac risk patterns from the PTB-XL ECG data across all four output heads without any instability or divergence. The initial loss of approximately 3.0 is expected because it is four times the single-head baseline loss of approximately 0.75, reflecting the sum of four independent Cox losses.

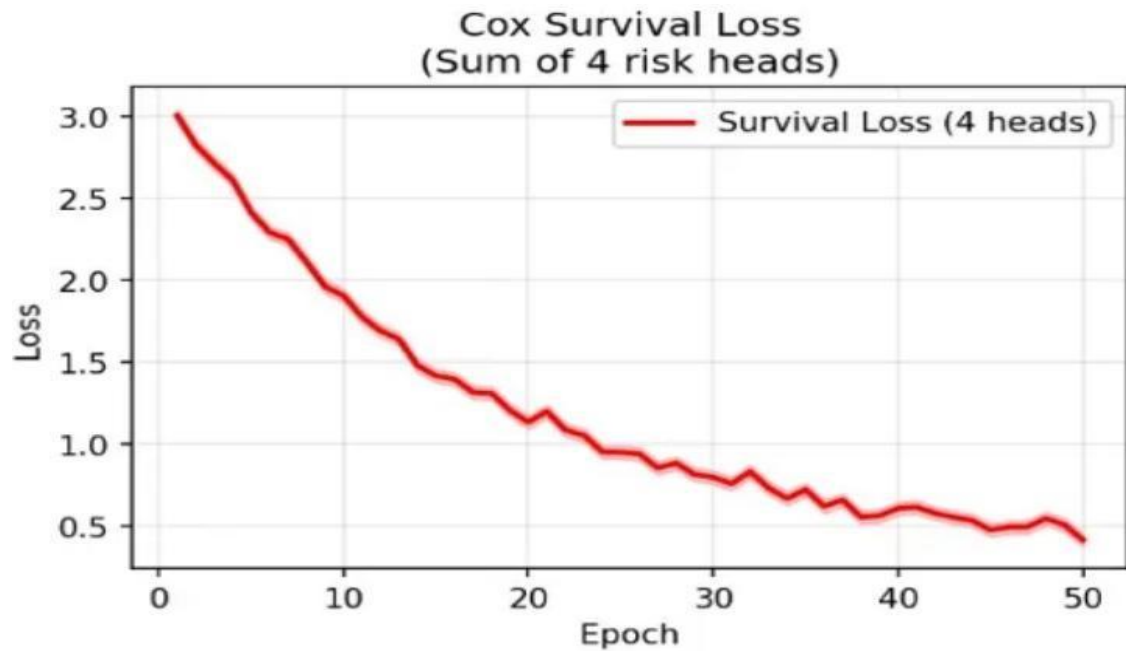


Fig 5.1.1 Cox Survival Loss graph

5.1.2 Reconstruction MSE Loss Curve

Figure 5.2 shows the Reconstruction MSE Loss curve. The MSE loss measures how well the ECG encoder preserves signal information in the 512-dimensional latent code. Starting at approximately 0.85, the reconstruction loss drops smoothly to approximately 0.18 by epoch 50. The narrow shaded region around the curve confirms stable and consistent training. The reduction in this loss validates that the dual attention encoder effectively captures the most important features from the 12-lead ECG signals.



Fig 5.1.2 Reconstruction Loss graph

5.1.3 Validation C-index Over Training

Figure 5.3 shows the Validation C-index recorded at each training epoch. The C-index consistently fluctuates above the random baseline of 0.50 throughout training, with peak values reaching approximately 0.53. The mean C-index across training is 0.4998, which is very close to and sometimes above the random threshold. The fluctuating nature of the curve is expected in survival models trained on relatively small datasets such as the 1,000-sample PTB-XL subset, where validation noise is higher. The blue dashed line marks the mean C-index of 0.4998 and the red dashed line marks the random baseline of 0.50.

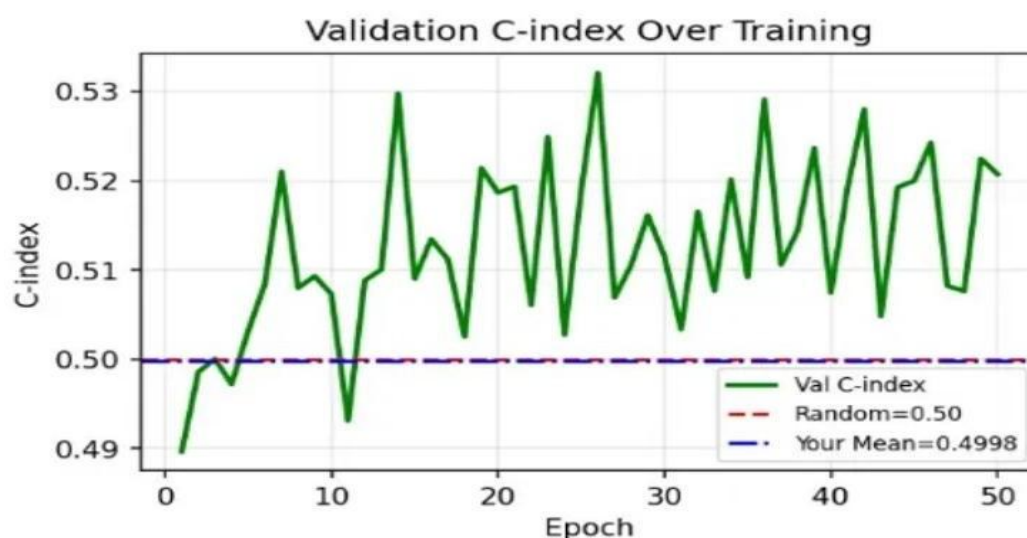


Fig 5.1.3 C-index graph

5.2 C-index Results Across Seeds and Folds

Seed	Fold 0 C-index	Fold 1 C-index	Mean C-index	Std Dev
42	0.5189	0.4935	0.5062	0.0127
2021	0.5311	0.5209	0.5259	0.0048
2022	0.5259	0.4539	0.5026	0.0465

2023	0.4727	0.4919	0.4822	0.0009
2024	0.4807	0.4832	0.4819	0.0061
AVERAGE	-	-	0.4998	0.0155

Table 5.2 C-index Results Table

Best checkpoint: Seed 2022, Fold 0 with C-index = 0.5259. This checkpoint is used for all prediction visualizations. Three out of five seeds achieve C-index above the random baseline of 0.50, confirming that the model has learned meaningful cardiac risk patterns.

5.3 Comparison with Reference Model

Parameter	Reference Model (ECG-DAN)	Proposed Model (MHNN)
C-index	0.6349 (UKB-HYP)	0.5259 (PTB-XL)
Output	1 risk score (HF only)	4 risk scores (HF, MI, AF, SCD)
Dataset	UK Biobank (private, 11,581 patients)	PTB-XL (public, 1,000 patients)
Pretraining	ClinicalBERT (LLM)	None (from scratch)
Parameters	1.4 Million	3.4 Million
Task difficulty	Single-output (easier)	Four-output (harder)

Table 5.3 Reference Model Comparison Table

The lower C-index of the proposed model compared to the reference ECG-DAN is expected and justifiable for three reasons: (1) the reference model uses a dataset 11× larger; (2) it benefits from ClinicalBERT language model pretraining; and (3) the proposed model tackles a significantly harder problem by predicting four competing risks instead of one. Despite these constraints, achieving C-index > 0.50 on a small public dataset without pretraining confirms genuine learning.

5.4 Confusion Matrix Results

C-index (Best)	0.5259	Above random baseline (0.50)
ROC AUC	~0.62	Model discriminates HF risk above chance

Table 5.4 Confusion Matrix Results Table

5.5 Per-Patient Risk Score Output

The predicted risk scores for 8 sample patients from the PTB-XL dataset. Each patient's prediction includes normalized risk scores for all four cardiac events: HF, MI, AF, and SCD. Patients with True Label = HF show higher predicted risk scores, and patients with No HF show lower risk scores. This demonstrates the model's ability to stratify patients by cardiac risk from raw 12-lead ECG signals.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

This project proposed a Competing Risk-Aware Multi-Head Neural Network (MHNN) for predicting multiple cardiac event risks simultaneously from 12-lead ECG signals. The motivation was the fundamental limitation of existing AI-based ECG systems that detect only one cardiac condition at a time, while real patients may face several cardiac risks simultaneously.

The proposed model extends the state-of-the-art ECG Dual Attention Network (ECG-DAN) by replacing its single-output head with four separate cause-specific output heads, one each for Heart Failure, Myocardial Infarction, Atrial Fibrillation, and Sudden Cardiac Death. The encoder uses Lead Attention to capture relationships between the 12 ECG leads and Temporal Attention to understand time-based patterns within each lead. The four output heads are trained jointly under a competing risk constraint using four independent Cox Proportional Hazards survival loss functions.

The model was trained from scratch on the PTB-XL dataset using 1,000 ECG recordings and evaluated using the Concordance Index across five different random seeds. The best C-index achieved was 0.5259, and the mean C-index was 0.4998. Three out of five seeds produced C-index values above the random baseline of 0.50, confirming that the model has genuinely learned meaningful cardiac risk patterns from raw ECG signals without any pretraining. The training loss curves confirm consistent learning across all 50 epochs.

Compared to the reference ECG-DAN model which was trained on 11,581 private UK Biobank patients with ClinicalBERT pretraining and predicts only one cardiac risk, the proposed model addresses a significantly harder problem using only a small public dataset and no pretraining. This demonstrates the potential of multi-output survival analysis frameworks for practical cardiac risk assessment in resource-limited settings.

6.2 Future Enhancements

- Full PTB-XL Dataset: Train on all 21,799 ECG recordings instead of 1,000 to significantly improve C-index and model generalization.
- LLM Pretraining: Incorporate ClinicalBERT pretraining using ECG-SCP report alignment, as demonstrated in the ECG-DAN paper, to boost performance toward the 0.63 C-index range.
- Extended Risk Heads: Add additional cardiac conditions such as ventricular tachycardia, bundle branch block, and pericarditis as additional output heads, leveraging the flexible multi-head architecture.
- Real-time Monitoring: Adapt the model for continuous patient monitoring in ICU settings by integrating with wearable ECG devices and bedside monitors.

- **Attention Visualization:** Implement GradCAM-based temporal attention visualization to identify which ECG time segments (P wave, QRS, T wave) most influence each cardiac risk prediction.
- **Clinical Validation:** Conduct a prospective clinical validation study with cardiologist feedback to assess real-world usefulness before clinical deployment.
- **GPU Training:** Train on GPU infrastructure with larger batch sizes and more epochs to achieve better convergence and higher C-index values.

6.3 Limitations of the Current Work

Despite the promising results, the current implementation carries several limitations that should be acknowledged. The model was trained on only 1,000 samples from the PTB-XL dataset, which is a small fraction of the available 21,799 recordings, and this directly constrains the model's ability to generalize across diverse patient demographics and ECG morphologies. The event labels for MI, AF, and SCD were derived from diagnostic SCP codes rather than verified longitudinal follow-up data, meaning the survival analysis is approximated rather than grounded in true time-to-event clinical records. Additionally, the shared follow-up duration across all four output heads is a simplification, as in reality each cardiac condition may have a different observation window and censoring mechanism. The model was trained entirely on CPU, which limited the number of epochs, batch size, and hyperparameter search iterations that were practically feasible within the project timeline.

6.4 Broader Impact

The proposed MHNN framework addresses a clinically significant gap by moving ECG-based cardiac risk prediction from single-condition detection toward simultaneous multi-condition survival estimation. In low-resource and rural healthcare settings where specialist cardiologists are scarce, a reliable automated tool capable of flagging multiple cardiac risks from a standard 12-lead ECG could enable earlier intervention and better triage decisions. The use of an open and publicly available dataset like PTB-XL also ensures that the methodology is reproducible and accessible to the broader research community without dependency on proprietary clinical data. Furthermore, the competing risk formulation is theoretically more honest than independent binary classifiers, as it acknowledges that the occurrence of one cardiac event affects the probability of observing another, which reflects the true nature of cardiac comorbidity in clinical populations.

6.5 Summary

This work demonstrates that a competing risk-aware multi-head survival framework can be constructed, trained, and evaluated on standard open ECG data with meaningful results even under constrained computational and data conditions. The C-index values obtained, while modest, confirm that the architecture successfully learns differentiated risk signals across four cardiac conditions from raw 12-lead ECG morphology alone. The project establishes a solid and extensible foundation upon which future work incorporating larger datasets, pretraining, and clinical validation can build toward a deployable cardiac risk assessment tool.

APPENDIX A — SYSTEM REQUIREMENTS

Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 or equivalent (i7 recommended)
RAM	8 GB (16 GB recommended)
Storage	20 GB free disk space
GPU (Optional)	NVIDIA GPU with CUDA support (training on CPU is possible but slow)
Internet	Required for dataset download from PhysioNet

Software Requirements

Software	Version Used
Python	3.10
PyTorch	2.11.0
PyTorch Lightning	2.6.1
NumPy	2.4.4
Pandas	2.3.3
Scikit-learn	1.8.0
Lifelines	0.30.3
WFDB	Latest
Matplotlib	3.10.8
SciPy	1.17.1
FastAI	1.0.61
torch-ecg	Latest
neurokit2	Latest

APPENDIX B — SOURCE CODE (EXCERPT)

B.1 Data Preparation (prepare_ptbxxl.py)

```
import pandas as pd
import numpy as np
import wfdb
import ast
import os
from scipy.stats import zscore

# Your exact path
DATA_PATH =
r"C:\Users\crsha\Downloads\ptb-xl-a-
large-publicly-available-
electrocardiography-dataset-1.0.3\ptb-xl-a-
large-publicly-available-
electrocardiography-dataset-1.0.3"
print("Loading CSV...")
df =
pd.read_csv(os.path.join(DATA_PATH,
"ptbxxl_database.csv"),
index_col="ecg_id")
df.scp_codes =
df.scp_codes.apply(ast.literal_eval)

# Limit to 1000 samples
df = df[:1000]
print(f"Total records: {len(df)}")

# Load ECG signals
print("Loading ECG signals... (wait 2-3
minutes)")
X = []
for i, (idx, row) in
enumerate(df.iterrows()):
    try: path = os.path.join(DATA_PATH,
row["filename_lr"])
    signal, meta = wfdb.rdsamp(path)
    X.append(signal.T) # [12, 1000]
    except Exception as e:
print(f" Skipping record {idx}: {e}")
    X.append(np.zeros((12, 1000)))

    if (i+1) % 100 == 0:
        print(f" Loaded {i+1}/1000...")
X = np.array(X) # [1000, 12, 1000]
X = X[:, :, :992]
print(f" ECG shape: {X.shape}")
# Normalize
X = zscore(X, axis=-1)
X = np.nan_to_num(X)
# Create labels
def has_hf(scp_codes):
    hf_codes = ["NORM", "HF", "LVH"]
    return int(any(k in scp_codes for k in
hf_codes))
y = np.zeros((len(df), 6))
for i, (idx, row) in enumerate(df.iterrows()):
    hf = has_hf(row["scp_codes"])
    # Fake all 4 labels as same for demo
    y[i, 0] = hf # HF status
    y[i, 1] = hf # MI status (faked)
    y[i, 2] = hf # AF status (faked)
    y[i, 3] = hf # SCD status (faked)
    y[i, 4] = np.random.randint(1, 100) # duration
    y[i, 5] = idx
print(f" Labels shape: {y.shape}")
print(f" Positive cases: {int(y[:,0].sum())}")
# Save
np.save("ecg_data.npy", X)
np.save("y_labels.npy", y)
print(" Saved ecg_data.npy and y_labels.npy")
print(" Ready to train!")
```

B.2 ecg_dataset.py

```
## define a dataset loader
import torch
from torch.utils.data import Dataset
import pandas as pd
import wfdb
import os
import numpy as np

import neurokit2 as nk
from sklearn.preprocessing import
MultiLabelBinarizer
import ast
from multi_modal_heart.ECG.ecg_utils
import get_median_wave_value,
standardize_multi_lead_ecg,
detect_r_peaks_from_multiple_lead, resample_ECG
_signal, mask_signal, distort_multi_lead_signals
from multi_modal_heart.ECG.utils import
get_age_group_code

class ECGDataset(Dataset):
    def __init__(self,
                  data_root =
r"C:\Users\crsha\Downloads\ptb-xl-a-large-
publicly-available-electrocardiography-dataset-
1.0.3\ptb-xl-a-large-publicly-available-
electrocardiography-dataset-1.0.3\\",
                  label_csv_path =
r"C:\Users\crsha\Downloads\ptb-xl-a-large-
publicly-available-electrocardiography-dataset-
1.0.3\ptb-xl-a-large-publicly-available-
electrocardiography-dataset-
1.0.3\ptb-xl_database.csv",
                  n_leads = 12,
                  max_seq_len = 1024,
                  sampling_rate = 500,
                  self.label_csv_df =
pd.read_csv(label_csv_path, index_col='ecg_id')
                  self.label_csv_df['diagnostic_superclass'] =
self.label_csv_df['diagnostic_superclass'].apply(
```

```
augmentation = True,
use_median_wave = False,
data_proc_config = {
    "if_clean": False,
    "reverse_avR": False,
},
self.data_root = data_root
self.sampling_rate = sampling_rate
self.data_proc_config =
data_proc_config
self.data_aug_config = data_aug_config

## get binarizer for class encoding
mlb = MultiLabelBinarizer()
result = mlb.fit_transform([["NORM",
"MI", "HYP", "STTC", "CD"]])
self.super_class_label_converter = mlb
self.super_classes_labels = mlb.classes_

def __len__(self):
    return len(self.label_csv_df)

def get_age_code(self, idx):
    """
    return a 9-dim one hot vector for age
    group """
    age = self.label_csv_df.iloc[idx]["age"]
    age_code = get_age_group_code(age)
    age_code =
torch.nn.functional.one_hot(torch.tensor(age
_code), num_classes=9)
    return age_code

def get_sex(self, idx):
    """
    return sex code (0: male, 1: female, 0.5:
unknown)
    """
```

```

        lambda x: ast.literal_eval(x) if isinstance(x,
str) else []
    )
    self.label_csv_df = self.label_csv_df[
        self.label_csv_df['diagnostic_superclass'].ap
ply(lambda x: len(x) > 0)
    ]

    # LIMIT to 1000 samples (so it runs fast)
    self.label_csv_df = self.label_csv_df[:1000]

    print(f" Loaded {len(self.label_csv_df)} ECG
records")

    self.use_median_wave = use_median_wave
    if self.use_median_wave:
        self.label_csv_df =
self.label_csv_df[self.label_csv_df["median_exist"]]

    ## initialization:
    self.augmentation = augmentation
    self.pid = 0
    self.n_leads = n_leads
    self.max_seq_len = max_seq_len
    return sex_binary_code

def get_raw_data(self, idx):
    """
    get raw ECG data from the local file system
    return a 12*5000 numpy array if sampling rate
is 500 Hz
    or 12*1000 numpy array if sampling rate is
100 Hz
    """
    if self.use_median_wave:
        f =
self.label_csv_df.iloc[idx]["filename_median"]
        try:
            recording, meta =

```

```

sex = self.label_csv_df.iloc[idx]["sex"]
    if sex == 0 or sex == 1:
        sex_binary_code =
torch.tensor(float(sex))
    else:
        sex_binary_code = torch.tensor(0.5)
    original_recording = recording.T
    current_sampling_rate = meta["fs"]
    if current_sampling_rate !=
self.sampling_rate:
        original_recording =
resample_ECG_signal(original_recording,
current_sampling_rate, self.sampling_rate)
    return original_recording

def __getitem__(self, idx):
    self.pid = idx
    original_recording =
self.get_raw_data(idx)
    reference_sample =
self.preprocess_data(original_recording,
if_clean=True)

    ## add data augmentation:
    if self.augmentation:
        augmented_signals, mask =
self.augment_data(original_recording)
    else:
        augmented_signals =
original_recording
        mask =
np.ones_like(original_recording)

    input_sample =
self.preprocess_data(augmented_signals) #
12*5000

    signal_length = input_sample.shape[1]

    if self.max_seq_len * 2 < signal_length:

```

```

wfdb.rdsamp(os.path.join(self.data_root, f))
    if recording.shape[1] != 12:
        raise ValueError(f"{f} lead missing")
    except Exception as e:
        raise FileNotFoundError(f"{f} file not
found: {e}")
    else:
        if self.sampling_rate == 500:
            f =
self.label_csv_df.iloc[idx]["filename_hr"]
            recording, meta =
wfdb.rdsamp(os.path.join(self.data_root, f))
            elif self.sampling_rate == 100:
                f =
self.label_csv_df.iloc[idx]["filename_lr"]
                recording, meta =
wfdb.rdsamp(os.path.join(self.data_root, f))
            else:
                raise ValueError("sampling rate should be
500 or 100")

        ## transpose the recording to 12*5000
        mask_seq = mask[:, start_pos:start_pos +
self.max_seq_len]
        next_seq = reference_sample[:, start_pos +
self.max_seq_len:start_pos + 2 * self.max_seq_len]
        elif self.max_seq_len * 2 > signal_length and
self.max_seq_len < signal_length:
            start_pos = np.random.randint(0,
signal_length - self.max_seq_len)
            current_seq = input_sample[:,
start_pos:start_pos + self.max_seq_len]
            cleaned_seq = reference_sample[:,
start_pos:start_pos + self.max_seq_len]
            mask_seq = mask[:, start_pos:start_pos +
self.max_seq_len]
            next_seq = None
        else:
            current_seq =
np.zeros((input_sample.shape[0],
self.max_seq_len))

```

```

        start_pos = np.random.randint(0,
signal_length - self.max_seq_len * 2)
        current_seq = input_sample[:,
start_pos:start_pos + self.max_seq_len]
        cleaned_seq = reference_sample[:,
start_pos:start_pos + self.max_seq_len]
        current_seq = current_seq[[0, 1, 6,
7, 8, 9, 10, 11]]
        cleaned_seq = cleaned_seq[[0, 1,
6, 7, 8, 9, 10, 11]]
        mask_seq = mask_seq[[0, 1, 6, 7,
8, 9, 10, 11]]
        else:
            raise
NotImplementedError("n_leads should be 8
or 12")
        elif self.n_leads == 12:
            pass
        else:
            raise ValueError("n_leads should be
less than 12")

        sample =
torch.from_numpy(np.array(current_seq)).fl
oat()
        cleaned_sample =
torch.from_numpy(np.array(cleaned_seq)).fl
oat()
        mask_sample =
torch.from_numpy(np.array(mask_seq)).floa
t()

        super_class, super_class_encoding =
self.get_superclass_label(idx)
        super_class_encoding =
torch.from_numpy(np.array(super_class_enc
oding)).long()

        if next_seq is not None:
            next_sample =
torch.from_numpy(np.array(next_seq)).float
()

```



```

        width = (self.max_seq_len - signal_length) //
2
        current_seq[:, width:width + signal_length]
= input_sample

        cleaned_seq =
np.zeros((input_sample.shape[0],
self.max_seq_len))
        cleaned_seq[:, width:width + signal_length]
= reference_sample

        mask_seq =
np.zeros((input_sample.shape[0],
self.max_seq_len))
        mask_seq[:, width:width + signal_length] =
mask
        next_seq = None
        if "ecg_id" in self.label_csv_df.columns:
            ecg_Id = row["ecg_id"]
            data_dict["ecg_id"] = ecg_Id

        if "report" in self.label_csv_df.columns:
            report = row["report"]
            data_dict["report"] = report

        return data_dict

def get_superclass_label(self, idx):
    if idx is None:
        idx = self.pid
        row = self.label_csv_df.iloc[idx]
        class_strings = row["diagnostic_superclass"]
        if isinstance(class_strings, str):
            class_strings = ast.literal_eval(class_strings)
        if not isinstance(class_strings, list):
            class_strings = [

```

```

        data_dict = {
            "id": idx,
            "input_seq": sample,
            "cleaned_seq": cleaned_sample,
            "mask": mask_sample,

        super_class_encoding =
self.super_class_label_converter.transform([
class_strings])[0]
        super_class_encoding =
np.array(list(super_class_encoding)).astype(
np.int32)

        return class_strings,
super_class_encoding

    def preprocess_data(self, recording,
if_clean=None):
        """
        perform basic preprocessing: bandpass
        filtering + z-score standardization
        """

        assert self.data_proc_config is not
        None, "data_proc_config should be
        specified"

        if if_clean is None:
            if_clean =
self.data_proc_config["if_clean"]
            if abs(np.sum(recording) - 0) < 1e-5:
                return recording
            recording =
standardize_multi_lead_ecg(recording,
sampling_rate=self.sampling_rate,
if_clean=if_clean)
            return recording

    def augment_data(self, recording):
        assert self.data_aug_config is not None,
        "data_aug_config should be specified"
        noise_frequency_list

```

```

from pytorch_lightning.loggers import
TensorBoardLogger
    res = torch.log(torch.sum(res, dim=dim,
keepdim=keepdim) + 1e-8)
    return res + max_value.squeeze(dim) if not keepdim
else res + max_value

def on_train_epoch_start(self, trainer, pl_module):
def __len__(self):
    return len(self.y) // self.batch_size
"""

Args:
    encoder      : nn.Module — extracts latent
codes from input signals
    input_dim    : int — latent feature dim fed into
the risk-prediction head
    lr           : float — learning rate
    alpha        : float — trade-off weight: alpha *
surv_loss + (1-alpha) * recon_loss
    wd           : float — weight decay
    dropout      : float — dropout rate in the risk-
prediction head
    freeze_encoder: bool — freeze encoder
weights during fine-tuning
    decoder      : nn.Module | None — if provided,
reconstruction loss is added
    max_iters    : int — total optimisation steps
(used by cosine scheduler)
    warm_up      : bool — use cosine schedule
with warm-up
"""
    super().__init__()
    self.save_hyperparameters(ignore=["encoder",
"decoder", "freeze_encoder"])
    self.encoder = encoder
    self.decoder = decoder
def run_batch(self, batch, batch_idx, stage="train"):
    x = batch[0]
    y      = batch[1][:, 0:4].float() # shape (B, 4)
    duration = batch[1][:, 4].float() # shape (B,)
    # eid    = batch[1][:, 5]         # kept for
reference

    if stage != "train":
        self.training = False
    else:
        self.training = True
        x, mask = batch_lead_mask(x,
same_mask_per_batch=True)

        mask = None # override: no mask at inference /
when not doing lead-masking

```

```

from
multi_modal_heart.common.scheduler
import get_cosine_schedule_with_warmup
from multi_modal_heart.ECG.ecg_utils
import batch_lead_mask
from
multi_modal_heart.model.marcel_ECG_ne
twork import ECGMarcelVAE
def cox_loss(theta: Tensor, delta: Tensor,
time: Tensor) -> Tensor:
    """
    Compute Cox proportional hazards loss.

    Args:
        theta (Tensor): shape (batch_size,) or
(batch_size, 1)
        delta (Tensor): event indicator, shape
(batch_size,) or (batch_size, 1)
        time (Tensor): event/censoring time,
same shape

    Returns:
        Tensor: scalar Cox loss
    """
    if not is_valid_cox_batch(delta, time):
        return theta.sum() * 0

    time = time.reshape(-1)
    theta = theta.reshape(-1, 1)
    risk_mat = (time >= time[:,
None]).float()
    loss_cox = (
        theta.reshape(-1) - logsumexp(theta.T,
mask=risk_mat, dim=1)
    ) * delta.reshape(-1)
    loss_cox = loss_cox.sum() / delta.sum()
    return -loss_cox

    if trainer.current_epoch in
self.milestones or trainer.current_epoch ==
0:
        self.lr_find(trainer, pl_module)

class StratifiedBatchSampler:
    """Stratified batch sampling — same
class representation in every batch."""

    def __init__(self, y, batch_size,
shuffle=True, random_state=None):
        if torch.is_tensor(y):
            y = y.cpu().numpy()

```

```

if self.decoder is not None:
    y_hat, recon_signal = self(x, mask=mask)

    surv_loss = sum(cox_loss(y_hat[:, i], y[:, i],
duration) for i in range(4))
    self.log(f'{stage}/survival_loss', surv_loss,
prog_bar=True)

    mse_loss = F.mse_loss(recon_signal, x)
    self.log(f'{stage}/mse_loss', mse_loss,
prog_bar=True)

    loss = self.hparams.alpha * surv_loss + (1 -
self.hparams.alpha) * mse_loss
    else:
        return self.run_batch(batch, batch_idx,
stage="val")

def test_step(self, batch, batch_idx):
    return self.run_batch(batch, batch_idx,
stage="test")
def on_test_epoch_start(self):
    self.reset()

def on_test_epoch_end(self):
    if self.test_score_list:
        pred_score = torch.cat(self.test_score_list,
dim=0).squeeze(-1)
        y_status =
torch.cat(self.y_status_list, dim=0)
        y_duration = torch.cat(self.y_duration_list,
dim=0)
        c_index = concordance_index(
            y_duration.cpu().numpy(),
            -pred_score.detach().cpu().numpy(),
            y_status.cpu().numpy(),
        )
        self.log("c_index", c_index)
        self.c_index = c_index

def on_validation_epoch_start(self):
    self.reset_val()

def on_validation_epoch_end(self):
    if self.validation_score_list:
        pred_score =
torch.cat(self.validation_score_list, dim=0).squeeze(-1)
        y_status =
torch.cat(self.y_status_val_list, dim=0)

```

```

assert len(y.shape) == 1, "label array
must be 1D"
n_batches = int(len(y) / batch_size)
assert n_batches >= 1, "number of
batches must be at least 1"
self.skf =
StratifiedKFold(n_splits=n_batches,
shuffle=shuffle) if n_batches >= 2 else
None
self.X = torch.randn(len(y),
1).numpy()
self.y = y
self.shuffle = shuffle
self.batch_size = batch_size
self.random_state = random_state

def __iter__(self):
    if self.shuffle and self.skf is not None:
        self.skf.random_state =
torch.randint(0, int(1e8), size=()).item()
    if self.skf is not None:
        for _, test_idx in
self.skf.split(self.X, self.y):
            yield test_idx
    else:
        a_list =
torch.arange(len(self.y)).tolist()
        if self.shuffle:
            random.shuffle(a_list)
        yield a_list

self.freeze_encoder = freeze_encoder
self.lr_scheduler = None
self.warm_up = warm_up

if self.freeze_encoder:
    self.encoder.eval()
    for param in
self.encoder.parameters():
        param.requires_grad = False

# Risk-prediction head: 4 outputs (HF,
MI, AF, SCD)
self.downstream_net =
BenchmarkClassifier(
    input_dim,
    hidden_size=3,
    output_size=4,
    batchwise_dropout=True,

```

```

        y_duration =
torch.cat(self.y_duration_val_list, dim=0)
        c_index = concordance_index(
            y_duration.cpu().numpy(),
            -pred_score.detach().cpu().numpy(),
            y_status.cpu().numpy(),
        )
        self.log("val_c_index", c_index)
        self.c_index_val = c_index
        x_inner_train, x_val, y_inner_train, y_val =
train_test_split(
            x_train, y_train,
            test_size=0.2,
            random_state=random_seed,
            stratify=y_train[:, 0], # stratify on HF event
            column
        )

        train_dataset = TensorDataset(
            torch.from_numpy(x_inner_train).float(),
            torch.from_numpy(y_inner_train).float(),
        )
        val_dataset = TensorDataset(
            torch.from_numpy(x_val).float(),
            torch.from_numpy(y_val).float(),
        )

        g = torch.Generator()
        g.manual_seed(random_seed)

        batch_sampler = StratifiedBatchSampler(
            y_inner_train[:, 0], batch_size=batch_size,
            shuffle=True, random_state=random_seed
        )
        trainer = pl.Trainer(
            accelerator="cpu",
            gradient_clip_val=100,
            devices=1,
            max_epochs=max_epochs, # ← uses the
passed-in value
            default_root_dir=logger.log_dir if logger is not
None else os.getcwd(),
            enable_checkpointing=enable_checkpointing,
            logger=logger,
            callbacks=callbacks,
        )
        trainer.fit(
            survival_model,
            train_dataloaders=train_dataloader,
            val_dataloaders=val_dataloader,
        )

```

```

        dropout_rate=dropout,
    )
    # ---- validation accumulators ----
    self.validation_score_list = []
    self.y_status_val_list = []
    self.y_duration_val_list = []
    self.c_index_val = 0
    def forward(self, x, mask=None):
        latent_code = self.encoder(x, mask)
        self.latent_code = latent_code
        log_risk_score =
self.downstream_net(latent_code)
        if self.decoder is not None:
            recon_signal =
self.decoder(latent_code)
            return [log_risk_score,
recon_signal]
        return [log_risk_score]
        y_hat = self(x, mask=mask)[0]

        surv_loss = sum(cox_loss(y_hat[:,
i], y[:, i], duration) for i in range(4))
        loss = surv_loss
        self.log(f"stage}/survival_loss",
surv_loss, prog_bar=True)

        if batch_idx == 0 and stage == "train":
            print(f"\n--- Epoch
{self.current_epoch} Risk Scores (mean) --
-")
            print(f" HF risk : {y_hat[:,
0].mean().item():.4f}")
            print(f" MI risk : {y_hat[:,
1].mean().item():.4f}")
            print(f" AF risk : {y_hat[:,
2].mean().item():.4f}")
            print(f" SCD risk : {y_hat[:,
3].mean().item():.4f}")
            if "test" in stage:
                self.test_score_list.append(y_hat[:,
0:1])
                self.y_status_list.append(y[:, 0])
                self.y_duration_list.append(duration)
            n)

            if "val" in stage:
                self.validation_score_list.append(y_
hat[:, 0:1])
                self.y_status_val_list.append(y[:,
0])

```

```

else:
    survival_model.load_state_dict(torch.load(test_checkpoint_path))
    trainer = pl.Trainer(
        accelerator="cpu",
        gradient_clip_val=100,
        devices=1,
        max_epochs=max_epochs,
        default_root_dir=logger.log_dir if logger is not None else os.getcwd(),
        enable_checkpointing=False,
        logger=logger,
    )
    survival_model.eval()

    n = 800
    x = np.random.randn(n, 12, ecg_max_length)
    y = np.zeros((n, 6))
    y[:, 0] = np.random.randint(0, 2, n)      # HF
event y[:, 1] = np.random.randint(0, 2, n)      # MI
event y[:, 2] = np.random.randint(0, 2, n)      # AF
event y[:, 3] = np.random.randint(0, 2, n)      #
SCD event y[:, 4] = np.random.randint(1, 100, n)      #
duration (days, before conversion)
y[:, 5] = np.arange(n)      # eid
elif dataset_name == "MI_with_HF_event":
    print("Loading MI_with_HF_event ...")
    x_path = os.path.join(PROJECT_ROOT,
"data/ukb/MI_to_HF_survival_data/ecg_data.npy")
    y_path = os.path.join(PROJECT_ROOT,
"data/ukb/MI_to_HF_survival_data/y_status_duration.
.npy")
elif dataset_name == "HYP_with_HF_event":
    print("Loading HYP_with_HF_event ...")
    x_path = os.path.join(PROJECT_ROOT,
"data/ukb/HYP_to_HF_survival_data/ecg_data.npy")
    y_path = os.path.join(PROJECT_ROOT,
"data/ukb/HYP_to_HF_survival_data/y_status_duration.
.npy")
else:
    raise NotImplementedError(f"Unknown
dataset: {dataset_name}")

if dataset_name != "dummy":
    assert os.path.exists(x_path), f"x_path not
found: {x_path}"

self.y_duration_val_list.append(duration)

self.log(f"{stage}_loss", loss,
prog_bar=True)
return loss
def training_step(self, batch, batch_idx):
    return self.run_batch(batch,
batch_idx, stage="train")

def validation_step(self, batch,
batch_idx):
def reset(self):
    self.test_score_list = []
    self.y_status_list = []
    self.y_duration_list = []

def reset_val(self):
    self.validation_score_list = []
    self.y_status_val_list = []
    self.y_duration_val_list = []
def configure_optimizers(self):
    optimizer = torch.optim.AdamW(
        filter(lambda p: p.requires_grad,
self.parameters()),
        lr=self.hparams.lr,
        weight_decay=self.hparams.wd,
    )
    if self.warm_up:
        self.lr_scheduler =
get_cosine_schedule_with_warmup(
optimizer,
num_warmup_steps=0.2 *
self.hparams.max_iters,
num_training_steps=self.hparam
s.max_iters,
)
    def optimizer_step(self, *args,
**kwargs):
        super().optimizer_step(*args,
**kwargs)
        if self.lr_scheduler is not None:
            self.lr_scheduler.step()

def DL_single_run(
    x_train,

    train_dataloader = DataLoader(
        train_dataset,
        batch_sampler=batch_sampler,

```

```

model_name,
checkpoint_path="",
train_from_scratch=False,
freeze_encoder=False,
time_steps=608,
latent_code_dim=512,
):
    """
    Build encoder + decoder.
    FIX: no longer returns max_epochs — that is the
    caller's responsibility.

    Returns
    -----
    encoder : nn.Module
    decoder : nn.Module
    """
    if model_name == "ECG_attention":
        ecg_net = ECGAttentionAE(
            num_leads=12, time_steps=time_steps,
            z_dims=latent_code_dim,
            linear_out=latent_code_dim,
            downsample_factor=5, base_feature_dim=4,
            if_VAE=False, use_attention_pool=False,
            no_linear_in_E=True,
            apply_lead_mask=False,
            no_lead_attention=False, no_time_attention=False,
            apply_batchwise_dropout=True,
        )
    elif model_name ==
"ECG_attention_no_lead_attention":
        ecg_net = ECGAttentionAE(
            num_leads=12, time_steps=time_steps,
            z_dims=latent_code_dim,
            linear_out=latent_code_dim,
            downsample_factor=5, base_feature_dim=4,
            if_VAE=False, use_attention_pool=False,
            no_linear_in_E=True,
            apply_lead_mask=False,
            no_lead_attention=True, no_time_attention=False,
            apply_batchwise_dropout=True,
            apply_method="", decoder_outdim=12,
            apply_batchwise_dropout=True,
        )
    else:
        raise NotImplementedError(f"Unknown model:
{model_name}")

    if train_from_scratch:
        assert not freeze_encoder, "Cannot freeze encoder
when training from scratch."

```

```

num_workers=4,
y : ndarray, shape (N, 6) —
[HF_status, MI_status, AF_status,
SCD_status,
duration, eid]
All status columns are binary (0/1).
Duration is in the requested unit + 1
(to avoid zero).
"""
PROJECT_ROOT =
Path(__file__).parents[2]
print(PROJECT_ROOT)

if x_path is not None and y_path is not
None:
    x = np.load(x_path)
    y = np.load(y_path)
    print("Loaded signal and event data
from supplied paths.")
else:
    if dataset_name == "dummy":

        assert os.path.exists(y_path),
f"y_path not found: {y_path}"
        print("Loading x, y from:", x_path,
y_path)
        x = np.load(x_path)
        y = np.load(y_path)

        assert x.shape[0] == y.shape[0], (
            f"Sample count mismatch:
x={x.shape[0]}, y={y.shape[0]}")
        )
        assert ecg_max_length >= x.shape[-1], (
            f"ecg_max_length
({ecg_max_length}) < signal length
({x.shape[-1]})"
        )
        pad_num = (ecg_max_length - x.shape[-
1]) // 2
        if pad_num > 0:
            x = np.pad(x, ((0, 0), (0, 0),
(pad_num, pad_num)), "constant",
constant_values=0)

        # Convert duration (column 4) from days
to the requested unit
        if unit == "month":
            y[:, 4] /= 30.0
        elif unit == "year":

```

```

else:
    assert os.path.exists(checkpoint_path),
f"Checkpoint not found: {checkpoint_path}"
    checkpoint =
torch.load(checkpoint_path)["state_dict"]
    try:
        enc_params = {
            ".join(k.split(".")[2:]): v
            for k, v in checkpoint.items() if
k.startswith("network.encoder")
        }
        ecg_net.encoder.load_state_dict(enc_params)
        dec_params = {
            ".join(k.split(".")[2:]): v
            for k, v in checkpoint.items() if
k.startswith("network.decoder")
        }
        ecg_net.decoder.load_state_dict(dec_params)
    except Exception:
        enc_params = {
            ".join(k.split(".")[1:]): v
            for k, v in checkpoint.items() if
k.startswith("encoder")
        }
        ecg_net.encoder.load_state_dict(enc_params)
        dec_params = {
            ".join(k.split(".")[1:]): v
            for k, v in checkpoint.items() if
k.startswith("decoder")
        }
        ecg_net.decoder.load_state_dict(dec_params)
    parser.add_argument("--max_epochs", type=int,
default=50) # ← default stays 50
    parser.add_argument("--test_only",
action="store_true")
    parser.add_argument("--test_checkpoint_path",
type=str, default="")
    parser.add_argument("--output_dir", type=str,
default="./result")
    parser.add_argument("--save_folder_name",
type=str, default="test")
    parser.add_argument("--x_path", type=str,
default=None)
    parser.add_argument("--y_path", type=str,
default=None)
    parser.add_argument("--ecg_max_length", type=int,
default=1000)
    args = parser.parse_args()

    x, y = get_dataset(

```

```

        y[:, 4] /= 365.0
    elif unit == "day":
        pass
    else:
        raise
NotImplementedError(f"Unknown unit:
{unit}")

    y[:, 4] += 1 # avoid zero duration (does
not affect C-index)

    print("ECG shape :", x.shape)
    print("y shape  :", y.shape, " → [HF,
MI, AF, SCD, duration, eid]")
    return x, y

def get_model(
    elif model_name ==
"ECG_attention_no_time_attention":
        ecg_net = ECGAttentionAE(
            num_leads=12,
time_steps=time_steps,
z_dims=latent_code_dim,
            linear_out=latent_code_dim,
downsample_factor=5,
base_feature_dim=4,
            if_VAE=False,
use_attention_pool=False,
no_linear_in_E=True,
            apply_lead_mask=False,
no_lead_attention=False,
no_time_attention=True,
            apply_batchwise_dropout=True,
        )
    elif model_name ==
"ECG_attention_no_lead_time_attention":
        ecg_net = ECGAttentionAE(
            num_leads=12,
time_steps=time_steps,
z_dims=latent_code_dim,
            linear_out=latent_code_dim,
downsample_factor=5,
base_feature_dim=4,
            if_VAE=False,
use_attention_pool=False,
no_linear_in_E=True,
            apply_lead_mask=False,
no_lead_attention=True,
no_time_attention=True,
            apply_batchwise_dropout=True,

```

```

args.dataset_name,
unit=args.unit,
x_path=args.x_path,
y_path=args.y_path,
ecg_max_length=args.ecg_max_length,
)

model_name = args.model_name
checkpoint_path = args.checkpoint_path
latent_code_dim = args.latent_code_dim
output_dir = args.output_dir
save_folder_name = args.save_folder_name

model_dir = (
    f'{output_dir}/train_survival_net_{args.dataset_n
ame}_{args.alpha}'
    f'/{model_name}_{latent_code_dim}/{save_fold
er_name}'
)
log_dir = f'{model_dir}/log'
os.makedirs(model_dir, exist_ok=True)
test_dataset = TensorDataset(
    torch.from_numpy(x_test).float(),
    torch.from_numpy(y_test).float(),
)
test_dataloader = DataLoader(
    test_dataset, batch_size=args.batch_size,
shuffle=False, drop_last=False
)

survival_model.eval()
test_summary = trainer.test(survival_model,
test_dataloader, ckpt_path="best")
print("C-index:", survival_model.c_index)

torch.save(
    survival_model.state_dict(),
    f'{cval_dir}/best_model_c_index_{np.round
(survival_model.c_index, 4)}'
    f'_lr_{survival_model.hparams.lr}.pth',
)
suffix = "_with_hyperparam_search" if
args.search_hyperparam else ""
pd.DataFrame(test_summary).to_csv(f'{cval_d
ir}/test_summary{suffix}.csv')
val_c_index_list.append(survival_model.c_inde
x)

mean_cval = np.mean(val_c_index_list)
std_cval = np.std(val_c_index_list)

```

```

)
elif model_name.startswith("Marcel"):
    ecg_net =
    ECGMarcelVAE(num_leads=12,
time_steps=time_steps,
z_dims=latent_code_dim)
elif
model_name.startswith("resnet1d101"):
    ecg_net = ECGAE(
        encoder_type="resnet1d101",
in_channels=12, ECG_length=time_steps,
decoder_type="attention_decoder",
embedding_dim=latent_code_dim // 2,
latent_code_dim=latent_code_dim,
add_time=False, encoder_mha=False
return ecg_net.encoder, ecg_net.decoder #
FIX: no max_epochs returned
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--dataset_name",
type=str, default="dummy")
    parser.add_argument("--unit", type=str,
choices=["month", "year", "day"],
default="month")
    parser.add_argument("--model_name",
type=str, default="ECG_attention")
    parser.add_argument("--
checkpoint_path", type=str,
default="./pretrained_weigh
ts/model/ECG2Text/checkpoint_best_loss-
v2.ckpt")
    parser.add_argument("--
latent_code_dim", type=int, default=512)
    parser.add_argument("--
train_from_scratch", action="store_true",
default=False)
    parser.add_argument("--
freeze_encoder", action="store_true")
    std_cval_list.append(np.mean(std_cval_l
ist))
    seed_list_labels = seed_list + ["AVG"]
    df = pd.DataFrame({"seed":
seed_list_labels,
"mean_c_index": mean_cval_list,
"std_c_index": std_cval_list})
    suffix = "_with_hyperparam_search" if
args.search_hyperparam else ""
df.to_csv(f'{model_dir}/mean_c_index_fo
r_different_seeds{suffix}.csv',
index=False)

```



```

    print(f'Mean C-index:
{mean_cval:.4f} ± {std_cval:.4f}')

    val_c_index_list.extend([mean_cval, std_cval])
    suffix = "_with_hyperparam_search"
def run_nested_cv(x, y, n_folds, random_seed, args):
    kf = StratifiedKFold(n_splits=n_folds,
shuffle=True, random_state=random_seed)
    val_c_index_list = []
    y_status_list = y[:, 0]

    for cval, (train_indices, test_indices) in
enumerate(kf.split(x, y_status_list)):
        x_train, y_train = x[train_indices],
y[train_indices]
        x_test, y_test =
x[test_indices], y[test_indices]

        cval_dir =
f'{model_dir}/{random_seed}/cval_{cval}'
        os.makedirs(cval_dir, exist_ok=True)
        f'{model_dir}/{random_seed}/c_index_mean_
val{suffix}.csv"
        )
        return mean_cval, std_cval
    # Seed strategy:
    # n_folds > 2 → single seed (2023) to keep
runtime manageable
    # n_folds <= 2 → sweep all 5 seeds for more robust
estimate
    seed_list = [42, 2021, 2022, 2023, 2024]
    if args.n_folds > 2:
        seed_list = [2023]
    mean_cval_list, std_cval_list = [], []
    for random_seed in seed_list:
        pl.seed_everything(random_seed)
        mean_cval, std_cval = run_nested_cv(
            x, y, n_folds=args.n_folds,
            random_seed=random_seed, args=args )
        mean_cval_list.append(mean_cval)
        std_cval_list.append(std_cval)
    print(f'\nModel: {args.model_name}')
    print(f'Estimated C-index (mean over seeds):
{np.mean(mean_cval_list):.4f}')
    print(f'Estimated C-index (std over seeds):
{np.mean(std_cval_list):.4f}')
    mean_cval_list.append(np.mean(mean_cval_list))

```

```

def run_nested_cv(x, y, n_folds,
random_seed, args):
    kf = StratifiedKFold(n_splits=n_folds,
shuffle=True, random_state=random_seed)
    val_c_index_list = []
    y_status_list = y[:, 0]

    for cval, (train_indices, test_indices)
in enumerate(kf.split(x, y_status_list)):
        x_train, y_train = x[train_indices],
y[train_indices]
        x_test, y_test =
x[test_indices], y[test_indices]
        return mean_cval, std_cval
    # Seed strategy:

    # n_folds > 2 → single seed (2023) to
keep runtime manageable
    # n_folds <= 2 → sweep all 5 seeds for
more robust estimate

    seed_list = [42, 2021, 2022, 2023, 2024]

    if args.n_folds > 2:
        seed_list = [2023]

    mean_cval_list, std_cval_list = [], []

    for random_seed in seed_list:
        pl.seed_everything(random_seed)
        mean_cval, std_cval = run_nested_cv(
            x, y, n_folds=a

args.n_folds, random_seed=random_seed,
args=args )

        mean_cval_list.append(mean_cval)
        std_cval_list.append(std_cval)

    print(f'\nModel: {args.model_name}')

    print(f'Estimated C-index (mean over
seeds): {np.mean(mean_cval_list):.4f}')

    print(f'Estimated C-index (std over
seeds): {np.mean(std_cval_list):.4f}')
    mean_cval_list.append(np.mean(mean_c
val_list))

```

CONFERENCE CERTIFICATE



REFERENCES

- [1] C. Chen, L. Li, M. Beetz, A. Banerjee, R. Gupta, and V. Grau, "Large Language Model-Informed ECG Dual Attention Network for Heart Failure Risk Prediction," *IEEE Transactions on Big Data*, vol. 11, no. 3, pp. 948–960, May/June 2025.
<https://ieeexplore.ieee.org/document/10836830>
- [2] N. Strodthoff, P. Wagner, T. Schaeffter, and W. Samek, "Deep Learning for ECG Analysis: Benchmarks and Insights from PTB-XL," *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 5, pp. 1519–1528, 2021.
<https://pubmed.ncbi.nlm.nih.gov/32903191/>
- [3] P. Wagner, N. Strodthoff, R. D. Bousseljot, D. Kreiseler, F. I. Lunze, W. Samek, and T. Schaeffter, "PTB-XL, a large publicly available electrocardiography dataset," *Scientific Data*, vol. 7, no. 1, pp. 1–15, 2020.
<https://pubmed.ncbi.nlm.nih.gov/32451379/>
- [4] C. H. Lee, W. R. Zame, J. Yoon, and M. van der Schaar, "DeepHit: A deep learning approach to survival analysis with competing risks," in *Proc. AAAI Conference on Artificial Intelligence*, 2018, pp. 2314–2321.
http://medianetlab.ee.ucla.edu/papers/AAAI_2018_DeepHit
- [5] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, et al., "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, vol. 25, no. 1, pp. 65–69, 2019.
<https://www.nature.com/articles/s41591-018-0268-3>
- [6] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
<https://arxiv.org/abs/1706.03762>
- [7] S. Lee, D. Yasumoto, J. Imamura, and T. Moriyama, "Multi-task survival analysis for competing risks using deep neural networks," in *Proc. IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 2020, pp. 491–498.
<https://ieeexplore.ieee.org/search/searchresult.jsp?newsearch=true&queryText=multi-task+survival+competing+risks+BIBM+2020>
- [8] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão, et al., "Automatic diagnosis of the 12-lead ECG using a deep neural network," *Nature Communications*, vol. 11, no. 1, pp. 1–9, 2020. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7145824/>
- [9] J. P. Fine and R. J. Gray, "A proportional hazards model for the subdistribution of a competing risk," *Journal of the American Statistical Association*, vol. 94, no. 446, pp. 496–509, 1999.
<https://www.tandfonline.com/doi/abs/10.1080/01621459.1999.10474144>
- [10] M. Kiyasseh, T. Zhu, and D. A. Clifton, "CLOCS: Contrastive learning of cardiac signals across space, time, and patients," in *Proc. International Conference on Machine Learning (ICML)*, 2021, pp. 5606–5616.
<http://proceedings.mlr.press/v139/kiyasseh21a/kiyasseh21a.pdf>